



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №12
«СОЗДАНИЕ РЕПОРТОВ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель работы: создать комплексный отчёт, объединяющий методы анализа данных (регрессия, классификация, кластеризация) в Python и R.

Задачи:

1. Провести EDA, корреляционный анализ, регрессию и кластеризацию.
2. Сформировать отчёты в форматах PDF/HTML.
3. Сравнить возможности Python и R для автоматизации отчётности.

РЕЗУЛЬТАТЫ РАБОТЫ

Аналитическая часть

Листинг 1 – EDA и корреляция в Python

```
# Листинг 1 – EDA и корреляция в Python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("mxmh_survey_results.csv")
df.dropna(inplace=True)

# Описательная статистика
print(df[["Age", "Anxiety", "Depression", "Hours per day"]].describe())

# Корреляционная матрица
corr_matrix = df[["Age", "Anxiety", "Depression", "Hours per day"]].corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Корреляционная матрица")
plt.show()
```

Листинг 2 – EDA и корреляция в R

```
# Листинг 2 – EDA и корреляция в R
library(corrplot)
df <- read.csv("mxmh_survey_results.csv")
df <- na.omit(df)

# Описательная статистика
summary(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])

# Корреляционная матрица
corr_matrix <- cor(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])
corrplot(corr_matrix, method="circle")
```

Визуализация матрицы корреляций в Python представлена на рис. 1.

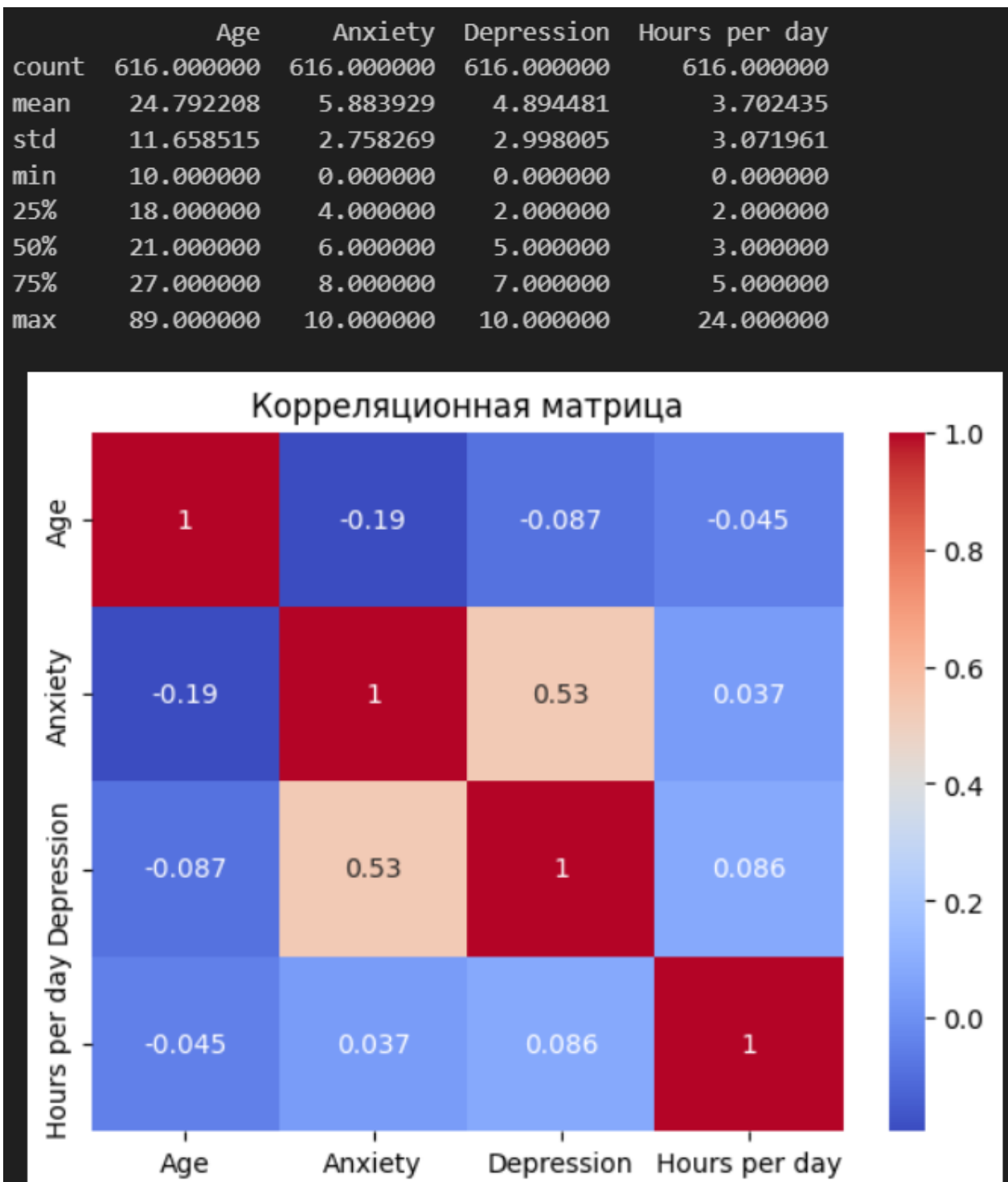


Рисунок 1 – Корреляционная матрица в Python

Визуализация матрицы корреляций в R представлена на рис. 2.

corrplot 0.95 loaded

Age	Anxiety	Depression	Hours.per.day
Min. :10.00	Min. : 0.000	Min. : 0.000	Min. : 0.000
1st Qu.:18.00	1st Qu.: 4.000	1st Qu.: 2.000	1st Qu.: 2.000
Median :21.00	Median : 6.000	Median : 5.000	Median : 3.000
Mean :24.76	Mean : 5.874	Mean : 4.887	Mean : 3.711
3rd Qu.:27.00	3rd Qu.: 8.000	3rd Qu.: 7.000	3rd Qu.: 5.000
Max. :89.00	Max. :10.000	Max. :10.000	Max. :24.000

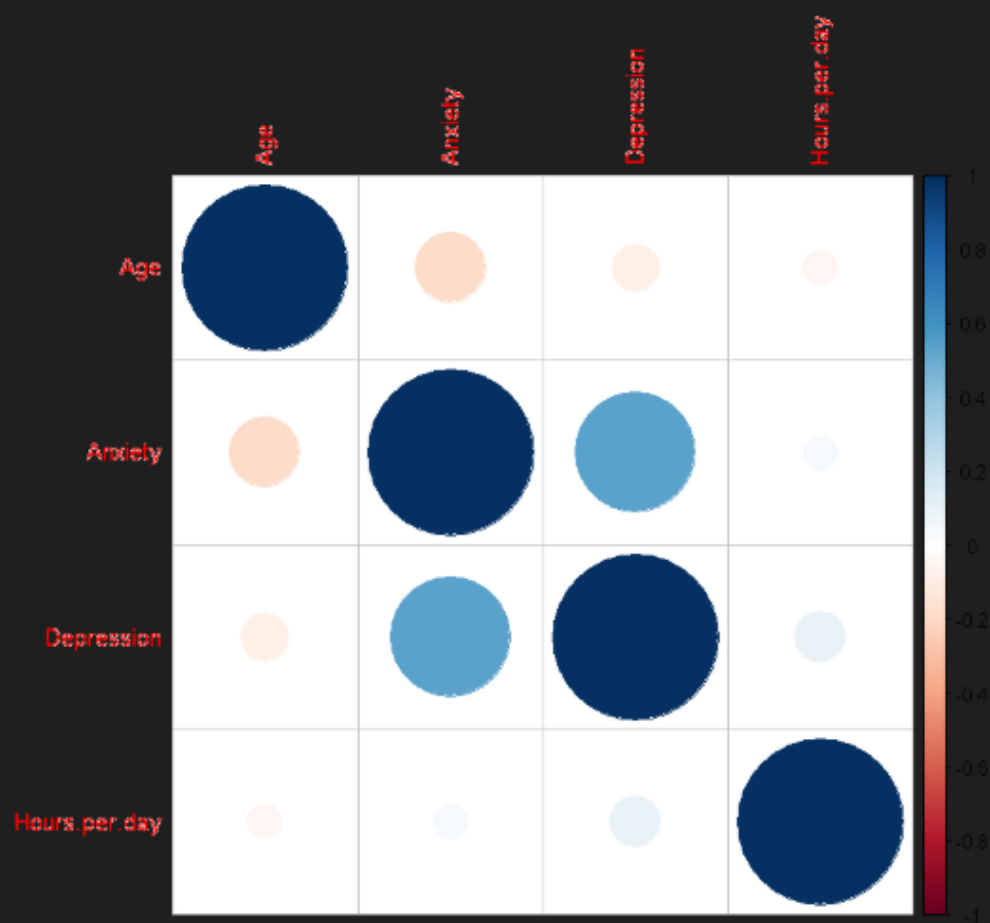


Рисунок 2 – Корреляционная матрица в R

Листинг 3 – Линейная регрессия в Python

```
# Листинг 3 – Линейная регрессия в Python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X = df[["Depression", "Hours per day", "Age"]]
y = df["Anxiety"]

model = LinearRegression().fit(X, y)
print(f"R²: {model.score(X, y):.2f}")
print(f"Коэффициенты: {model.coef_}")
```

Листинг 4 – Линейная регрессия в R

```
# Листинг 4 – Линейная регрессия в R
model <- lm(Anxiety ~ Depression + Hours.per.day + Age, data=df)
summary(model)
```

Веса модели в Python представлены на рис. 3.

```
R²: 0.30
Коэффициенты: [ 0.47482376 -0.01249656 -0.03532374]
```

Рисунок 3 – Summary модели в Python

Веса модели в R представлены на рис. 4.

```
Call:
lm(formula = Anxiety ~ Depression + Hours.per.day + Age, data = df)

Residuals:
    Min       1Q   Median       3Q      Max
-7.7547 -1.4956  0.1875  1.4624  6.1696

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   4.379012   0.293613   14.914 < 2e-16 ***
Depression     0.478968   0.031064   15.419 < 2e-16 ***
Hours.per.day -0.012752   0.030160   -0.423  0.673
Age           -0.032270   0.007959   -4.055 5.65e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.32 on 625 degrees of freedom
Multiple R-squared:  0.3,    Adjusted R-squared:  0.2966
F-statistic: 89.28 on 3 and 625 DF,  p-value: < 2.2e-16
```

Рисунок 4 – Веса модели в R

Листинг 5 -Логистическая регрессия в Python

```
# Листинг 5 – Логистическая регрессия в Python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X = pd.get_dummies(df[["Age", "Anxiety", "Depression"]])
y = df["Music effects"]

model = LogisticRegression(max_iter=1000).fit(X, y)
print(classification_report(y, model.predict(X)))
```

Листинг 6 – Логистическая регрессия в R

```
# Листинг 6 – Логистическая регрессия в R
library(caret)
library(nnet)
# Удаление столбца Timestamp
if ("Timestamp" %in% colnames(df)) {
  df$Timestamp <- NULL
}

# Кодирование категориальных признаков
cat_columns <- names(df)[sapply(df, is.character)]
for (col in cat_columns) {
  # Преобразуем в фактор с сохранением порядка уровней
  df[[col]] <- factor(df[[col]])
  # Кодируем в числовой формат
  df[[col]] <- as.numeric(df[[col]])
}

# Заполнение пропусков в числовых столбцах
numeric_columns <- names(df)[sapply(df, is.numeric)]
for (col in numeric_columns) {
  df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
}

# Преобразуем целевую переменную в фактор
df$Music.effects <- as.factor(df$Music.effects)

# Подготовка данных для классификации
X_class <- df[, !names(df) %in% c("Music.effects", "Anxiety", "Depression")]
y_class <- df$Music.effects
temp_class <- cbind(X_class, Music.effects = y_class)
set.seed(42)
trainIndex_class <- createDataPartition(temp_class$Music.effects, p = 0.7, list =
FALSE)
train_class <- temp_class[trainIndex_class, ]
test_class <- temp_class[-trainIndex_class, ]
model <- multinom(Music.effects ~ ., data = train_class)

# Прогнозирование на тестовых данных
pred_class <- predict(model, newdata = test_class)

# Преобразуем предсказания и тестовые данные в факторы с одинаковыми уровнями
levels_test <- levels(test_class$Music.effects)
pred_class <- factor(pred_class, levels = levels_test)
actual_class <- factor(test_class$Music.effects, levels = levels_test)

conf_matrix <- confusionMatrix(pred_class, actual_class)
print(conf_matrix)
```

Метрики модели в Python представлены на рис. 5.

	precision	recall	f1-score	support
Improve	0.76	1.00	0.86	465
No effect	1.00	0.01	0.01	136
Worsen	0.00	0.00	0.00	15
accuracy			0.76	616
macro avg	0.59	0.34	0.29	616
weighted avg	0.79	0.76	0.65	616

Рисунок 5 – Метрики модели в Python

Метрики модели в R представлены на рис. 6.

	Reference			
Prediction	1	2	3	4
1	0	0	0	0
2	1	138	38	3
3	0	3	3	1
4	0	0	0	0

Overall Statistics

Accuracy : 0.754
95% CI : (0.6859, 0.8139)
No Information Rate : 0.754
P-Value [Acc > NIR] : 0.5395

Kappa : 0.0753

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: 1	Class: 2	Class: 3	Class: 4
Sensitivity	0.000000	0.97872	0.07317	0.00000
Specificity	1.000000	0.08696	0.97260	1.00000
Pos Pred Value	NaN	0.76667	0.42857	NaN
Neg Pred Value	0.994652	0.57143	0.78889	0.97861
Prevalence	0.005348	0.75401	0.21925	0.02139
Detection Rate	0.000000	0.73797	0.01604	0.00000
Detection Prevalence	0.000000	0.96257	0.03743	0.00000
Balanced Accuracy	0.500000	0.53284	0.52289	0.50000

Рисунок 6 – Метрики модели в R

Листинг 7 – K-means кластеризация

```
# Листинг 7 – K-means кластеризация
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X = df[["Anxiety", "Depression", "Hours per day"]]
kmeans = KMeans(n_clusters=3).fit(X)
df["Cluster"] = kmeans.labels_

# Визуализация
plt.scatter(df["Anxiety"], df["Depression"], c=df["Cluster"], cmap="viridis")
plt.title("Кластеры пользователей")
plt.show()
```

Листинг 8 – K-means кластеризация в R

```
# Листинг 8 – K-means кластеризация в R
set.seed(42)
kmeans_result <- kmeans(df[, c("Anxiety", "Depression", "Hours.per.day")],
centers=3)
df$Cluster <- kmeans_result$cluster

# Визуализация
library(ggplot2)
ggplot(df, aes(Anxiety, Depression, color=as.factor(Cluster))) + geom_point()
```

Визуализация кластеров на Python представлена на рис. 7.

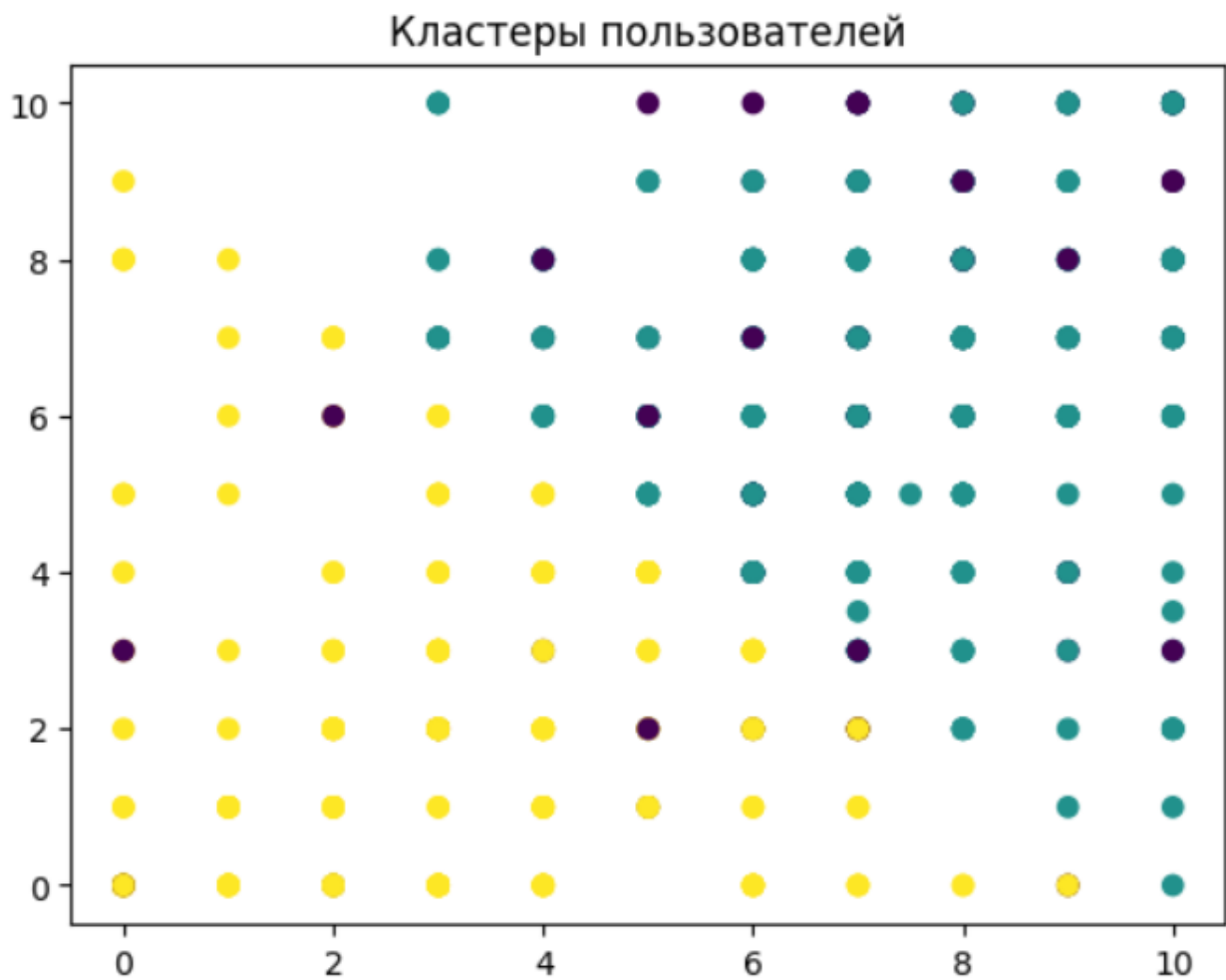


Рисунок 7 – Визуализация кластеров в Python

Визуализация кластеров на R представлена на рис. 8.

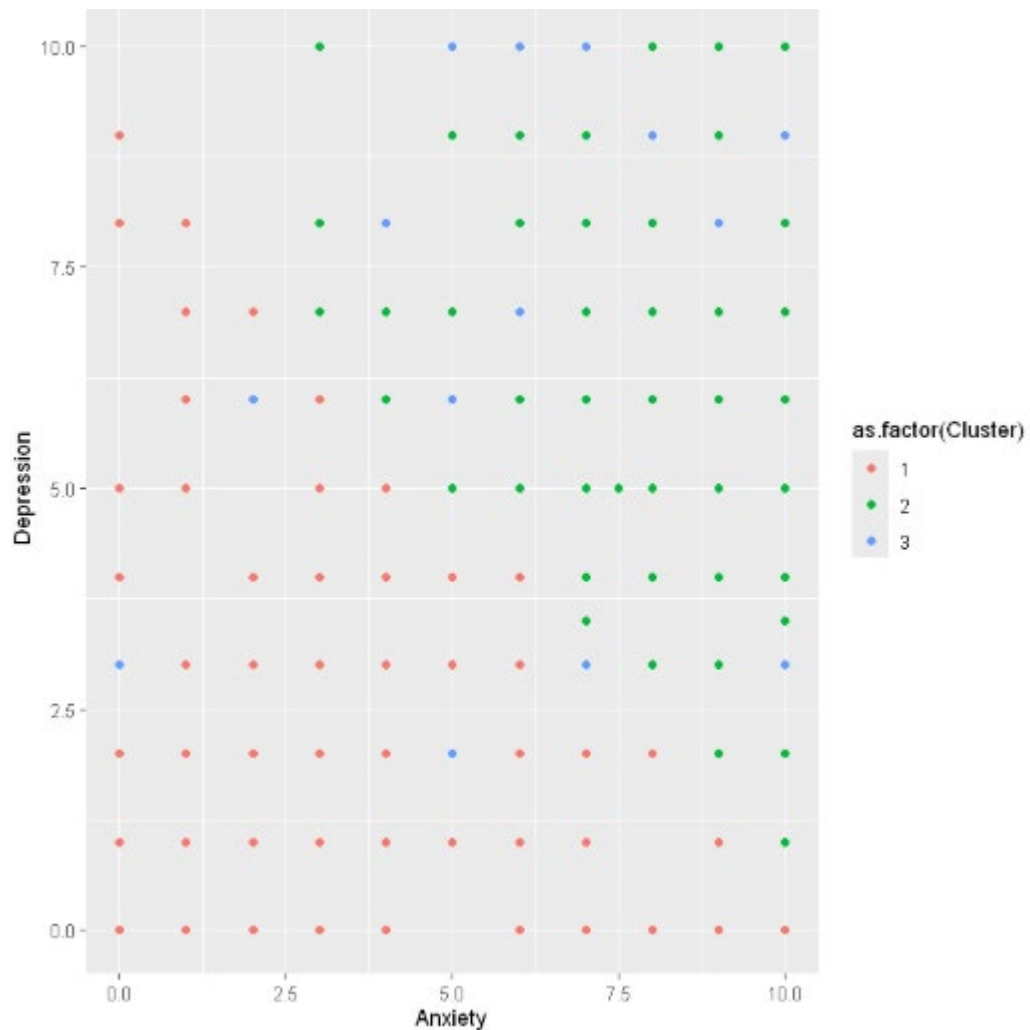


Рисунок 8 – Визуализация кластеров в R

Создание отчётов

Листинг 9 – Код для создания репорта в Python

```
!jupyter nbconvert --to pdf Япсод12_Py.ipynb
```

Листинг 10 – Код для Rmarkdown

```
---
---
title: "Анализ музыкальных предпочтений"
author: "Боргачев Тимофей"
date: "`r Sys.Date()`"
output:
  pdf_document:
    toc: true
    number_sections: true
    fig_caption: yes
    latex_engine: xelatex
---

```{r setup, include=FALSE}
Настройки knitr и глобальные опции
knitr::opts_chunk$set(
 echo = TRUE,
```

```

dev = "cairo_pdf",
message = FALSE,
warning = FALSE,
cache = TRUE,
fig.align = "center"
)
options(stringsAsFactors = FALSE)
Подключение необходимых библиотек
library(ggplot2)
library(corrplot)
library(caret)
library(nnet)
```

## EDA

```{r load-preprocess}
library(corrplot)
df <- read.csv("mxmh_survey_results.csv")
df <- na.omit(df)

Описательная статистика
summary(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])
```

## Корреляционный анализ

```{r correlation-plot, fig.width=6, fig.height=6}
Корреляционная матрица
corr_matrix <- cor(df[, c("Age", "Anxiety", "Depression", "Hours.per.day")])
Визуализация корреляций
corrplot(corr_matrix, method = "circle", type = "upper",
 addCoef.col = "black", tl.col = "black", tl.cex = 0.8)
```

## Линейная регрессия

```{r linear-model-summary}
Построение модели линейной регрессии
model_lm <- lm(Anxiety ~ Depression + Hours.per.day + Age, data = df)

Вывод результатов модели
summary(model_lm)
```

## Многоклассовая логистическая регрессия

```{r logistic-regression}
library(caret)
library(nnet)
Удаление столбца Timestamp
if ("Timestamp" %in% colnames(df)) {
 df$Timestamp <- NULL
}

```

```

}

Кодирование категориальных признаков
cat_columns <- names(df)[sapply(df, is.character)]
for (col in cat_columns) {
 # Преобразуем в фактор с сохранением порядка уровней
 df[[col]] <- factor(df[[col]])
 # Кодировем в числовой формат
 df[[col]] <- as.numeric(df[[col]])
}

Заполнение пропусков в числовых столбцах
numeric_columns <- names(df)[sapply(df, is.numeric)]
for (col in numeric_columns) {
 df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
}

Преобразуем целевую переменную в фактор
df$Music.effects <- as.factor(df$Music.effects)

Подготовка данных для классификации
X_class <- df[, !names(df) %in% c("Music.effects", "Anxiety", "Depression")]
y_class <- df$Music.effects
temp_class <- cbind(X_class, Music.effects = y_class)
set.seed(42)
trainIndex_class <- createDataPartition(temp_class$Music.effects, p = 0.7, list = FALSE)
train_class <- temp_class[trainIndex_class,]
test_class <- temp_class[-trainIndex_class,]
model <- multinom(Music.effects ~ ., data = train_class)

Прогнозирование на тестовых данных
pred_class <- predict(model, newdata = test_class)

Преобразуем предсказания и тестовые данные в факторы с одинаковыми уровнями
levels_test <- levels(test_class$Music.effects)
pred_class <- factor(pred_class, levels = levels_test)
actual_class <- factor(test_class$Music.effects, levels = levels_test)

conf_matrix <- confusionMatrix(pred_class, actual_class)
print(conf_matrix)
```



## ## Кластерный анализ



```

```{r clustering, fig.width=6, fig.height=5}
#Кластеризация методом K-means
set.seed(42)
kmeans_result <- kmeans(df[, c("Anxiety", "Depression", "Hours.per.day")],
centers=3)
df$Cluster <- kmeans_result$cluster

```


```

```
Визуализация
library(ggplot2)
ggplot(df, aes(Anxiety, Depression, color=as.factor(Cluster))) + geom_point()
```



```
```{r save-model, include=FALSE}
# Сохранение обученной модели многоклассовой регрессии
saveRDS(model, "music_effects_model.rds")
```
```


```

Пример реализованного отчета в Python представлен на рис. 9.

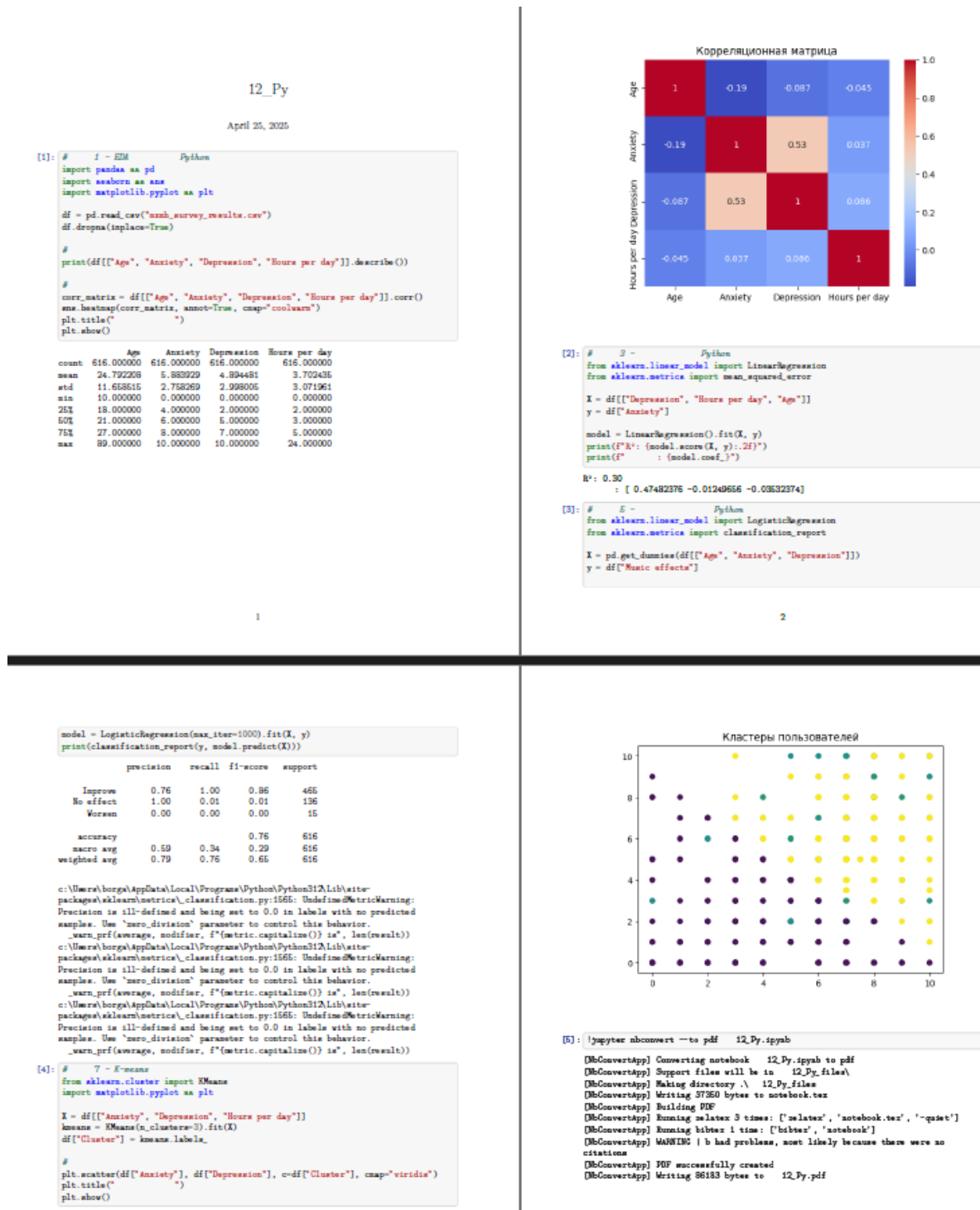


Рисунок 9 – Отчет в Python

Пример реализованного отчета в R представлен на рис. 10.

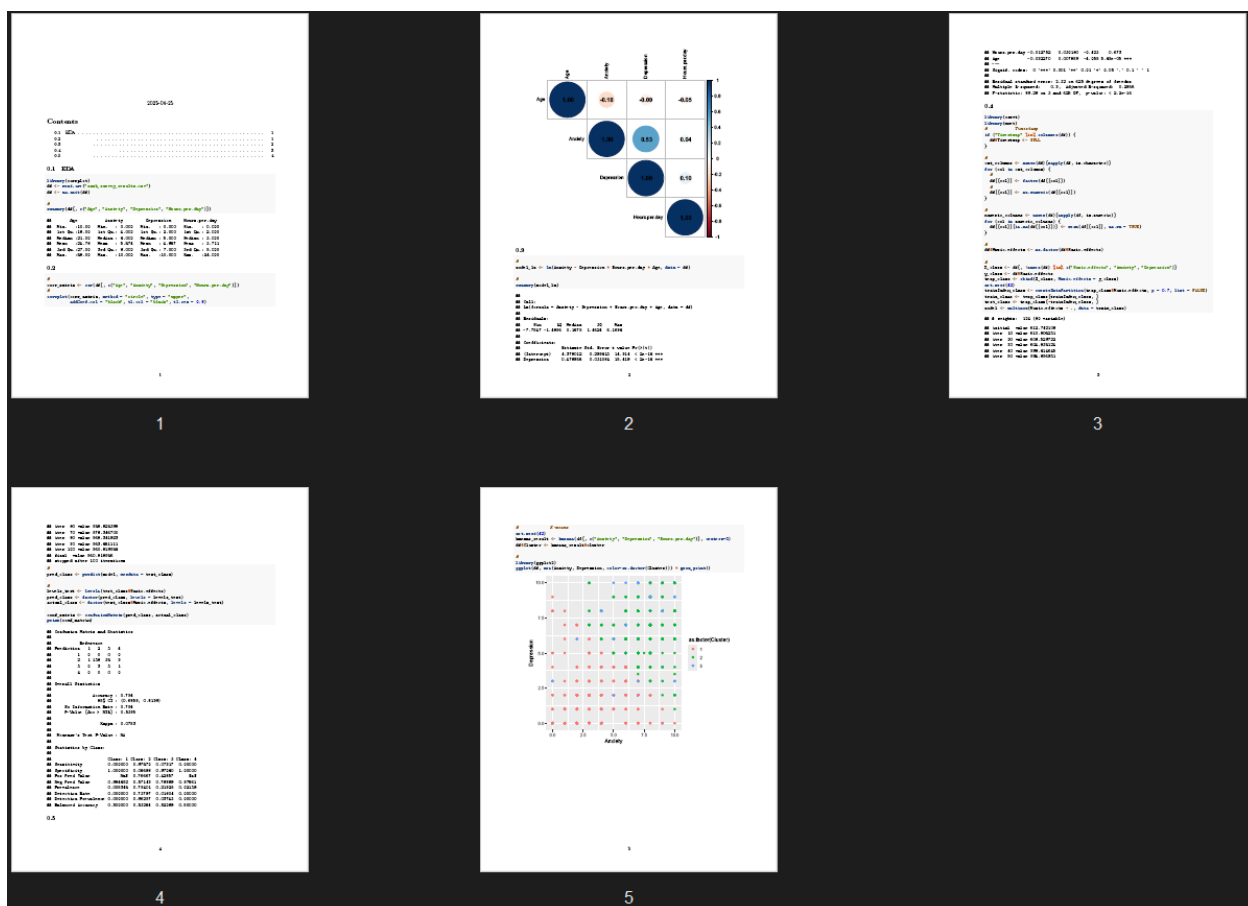


Рисунок 10 – Отчет в R

Сравнение Python и R

Сравнение инструментов представлено в табл. 1.

Таблица 1 – Сравнение инструментов создания отчетов

Параметр	Python	R
Создание отчётов	Гибкость Jupyter/Markdown	через RMarkdown (Knit → PDF/HTML)
Интерактивность	Plotly, Dash	Shiny, plotly
Производительность	Dask для больших данных	data.table

ЗАКЛЮЧЕНИЕ

В рамках выполненной работы были успешно реализованы задачи по созданию комплексного аналитического отчета с использованием Python и R. Проведен полный цикл анализа данных, включающий:

1. Исследовательский анализ данных (EDA) и корреляционный анализ
2. Построение моделей линейной и логистической регрессии
3. Выполнение кластеризации методом K-means
4. Создание автоматизированных отчетов в форматах PDF/HTML

В результате сравнения возможностей Python и R для создания отчетности выявлено:

- Python демонстрирует большую гибкость при работе с различными форматами отчетов через Jupyter Notebook и Markdown
- R предоставляет более интегрированное решение с помощью RMarkdown
- Оба языка имеют сопоставимые возможности для создания интерактивных элементов и обработки больших объемов данных

Работа показала, что выбор инструмента зависит от конкретных задач проекта и предпочтений аналитика. Созданные отчеты могут быть использованы как шаблоны для дальнейшего анализа подобных наборов данных.

Полученные результаты и разработанные скрипты могут служить основой для построения более сложных аналитических решений и автоматизации процессов формирования отчетности.